

Промышленная разработка ML

01

От эксперимента в ноутбуке к проверяемому Python-компоненту.

Данные

Код

Проверки

Доставка

О преподавателе

Егор Колодин



Опыт

Gradient — Senior CV Engineer

Сейчас

Sber GigaChat — Lead NLP Engineer

Образование

МФТИ · ШАД Яндекса

Связь

Telegram: @ekolodin

Карта курса: восемь тем

Сначала воспроизводимость и процесс, затем ускорение и интеграция.

01. Основы MLOps

Инструментарий и
передача результата

02. Код и данные

Стандартизация и
версионирование

03. Эксперименты и метрики

Трекинг и мониторинг

04. ML-процессы

Автоматизация и
масштабирование

05. NVIDIA GPU

Архитектура и
оптимизация

06. ONNX и TensorRT

Интеграция и
оптимизация

07. NVIDIA Triton

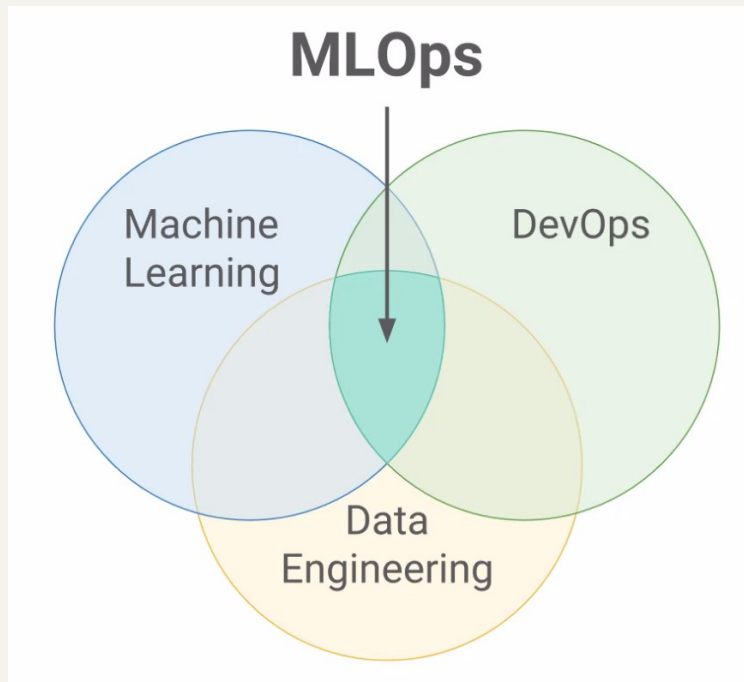
Inference Server

08. Финальный проект

Веб-интеграция и HNSW

Зачем MLOps, если модель уже обучена?

MLOps связывает разработку модели, данные и эксплуатацию.



ML

Что измеряем и с чем сравниваем?

Данные

Что было доступно в момент прогноза?

Эксплуатация

Как запустить, проверить и обновить?

Taxi-кейс: сначала контракт задачи

Предсказываем длительность до начала поездки; назначение указано пользователем.

Часть	Что передаём	Что проверяем
Вход	PULocationID, DOLocationID	Зоны известны заранее
Служебное поле	ride_id	Связь запроса и ответа
Выход	predicted_duration	Минуты; одна строка на вход
Target	dropoff – pickup	Только обучение и оценка

Утечка: признак пришёл из будущего

Для каждого поля задаём вопрос: откуда оно известно в момент прогноза?

Признак	Когда известен	Решение
Зона отправления	При запросе	Используем
Зона назначения	Если сообщил пользователь	Явное допущение
Итоговая стоимость	После поездки	Исключаем
Время окончания	После поездки	Только для target

Разделяем данные по времени

Green Taxi, 2021: по 10 000 строк на месяц до фильтра длительности.

01

Январь → train

Fit признаков и модели;
среднее для baseline



02

Февраль → validation

Только transform и оценка;
не отдельный test



03

Манифест

Источник, seed 2026,
SHA-256 файлов

Категории: fit один раз, transform дальше

Номер зоны — категория, а не численное расстояние между районами.

Строковые ID

74 → "74"; пропуск → "-1"

Train

Создаёт словарь признаков

Validation и запрос

Используют тот же словарь

PYTHON

```
vectorizer = DictVectorizer(sparse=True)
x_train = vectorizer.fit_transform(
    prepare_features(train))
x_valid = vectorizer.transform(
    prepare_features(validation))
model.fit(x_train, train.duration)
```

Качество: baseline на той же validation

RMSE измеряется в минутах; ниже — лучше. Сравниваем на одинаковых строках.

Вариант	На чём обучен	Validation RMSE
Среднее значение	Средняя duration января	≈ 12.14 мин
LinearRegression	Зоны января	≈ 10.66 мин
Вывод	Лучше baseline на этой когорте	Не разрешение на релиз

Где заканчиваются выводы эксперимента

Описываем не только метрику, но и область, к которой она относится.

01

Когорта

Duration 1–60 минут
включительно — только
для обучения и оценки

02

Период

Два месяца 2021 года;
историческая учебная
выборка

03

Применение

На входе нет target;
строки запроса не
фильтруем

Окружение: что фиксирует каждый файл

Python 3.13 и env готовим до практики.

venv

Изолирует установленные пакеты

pyproject.toml

Метаданные, сборка, допустимые зависимости

requirements-lock.txt

Проверенный набор точных версий

BASH

```
python3.13 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-lock.txt
python -m pip check
```

Репозиторий: код, тесты и автоматизация

У каждого инструмента отдельная ответственность.

Git

История изменений кода

Formatter и linter

Стиль и статические проверки

pre-commit и CI

Локальный запуск и общая проверка

TEXT

```
starter/  
  pyproject.toml  
  src/taxi_duration/  
    data.py  
    model.py  
    cli.py  
  tests/           # ваши тесты  
../tests/         # выданный контракт
```

Из ноутбука — в функции с границами

Чистые преобразования проще проверить, чем ячейку с чтением файлов и обучением.

■ data.py

Признаки и размеченная когорта

■ model.py

Обучение и предсказание

■ cli.py

Аргументы, файлы, ошибки
пользователя

PYTHON

```
records = prepare_features(frame)
train = prepare_training_data(history)
bundle, metrics = train_model(train, valid)
predictions = predict(bundle, requests)
```

CLI: два явных действия

Один интерфейс для человека, скрипта и автоматической проверки.

train

Данные → bundle, метрики,
паспорт

predict

Bundle + запросы → CSV

Повтор запуска

Новый каталог результата; не
затираем старый

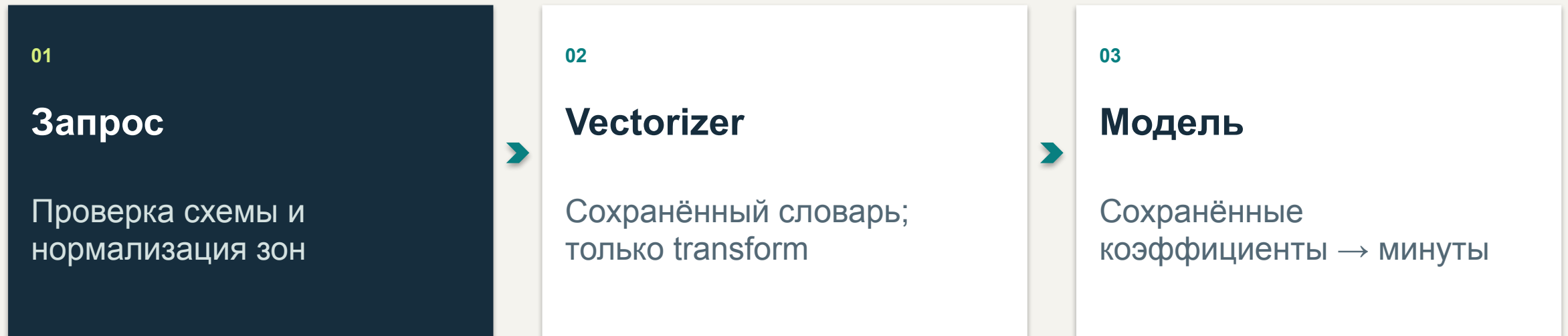
BASH

```
taxi-duration train \  
  --train ../data/train.parquet \  
  --validation ../data/validation.parquet \  
  --output artifacts
```

```
taxi-duration predict \  
  --model artifacts/model.pkl \  
  --input ../data/inference.csv \  
  --output predictions.csv
```

Сохраняем модель вместе с преобразователем

Обученные коэффициенты имеют смысл только в том же пространстве признаков.



Паспорт запуска: откуда взялся результат

Метрика без происхождения не позволяет сравнивать запуски.

Вход

Хеши данных и конфигурация

Исполнение

Версии среды, команда, хеши
кода

Выход

Модель, метрики и их хеши

TEXT

```
artifacts/  
  model.pkl          # fitted state  
  metrics.json       # RMSE, counts  
  run.json           # provenance  
  
# Код доставляется отдельно:  
dist/  
  taxi_duration_course-0.1.0-...whl
```

Unit-тест: маленькие данные, точный ответ

Тестируем границы и запреты, а не только типичный пример.

Граница когорты

59 с — нет; 60 с и 3600 с — да

Вход

Нет target; пропуск зоны → -1

Инварианты

Не меняем вход и порядок строк

PYTHON

```
frame = pd.DataFrame({
    "PULocationID": [1, None],
    "DOLocationID": [2, 3],
})
assert prepare_features(frame) == [
    {"PULocationID": "1", "DOLocationID": "2"},
    {"PULocationID": "-1", "DOLocationID": "3"},
]
```

Smoke и integration: работает ли путь?

Smoke — быстрый сигнал целостности; integration — проверка взаимодействий.

Проверка	Пример	Чего не доказывает
Smoke	Загрузить bundle; один predict	Качество на реальных данных
Integration	CLI train → файлы → CLI predict	Что wheel полон
Проверка доставки	Установить wheel вне исходников	Что модель полезна продукту

Регрессия: что изменилось после правки?

Разделяем сохранение поведения кода и допустимое изменение качества модели.

■ Инженерная регрессия

Save/load сохраняет ответы

■ ML-регрессия

Метрика на фиксированной выборке

■ Правило сравнения

Допуск и порог заданы заранее

PYTHON

```
before = predict(bundle, requests)
save_model(bundle, path)
restored = load_model(path)
after = predict(restored, requests)
np.testing.assert_allclose(
    before, after, rtol=1e-10, atol=1e-10)
```

Нагрузка и stress: где предел системы?

Корректный большой batch ещё не означает выполненный SLA.

Режим	Что меняем	Что наблюдаем
Batch-проверка	10 000 строк	Количество и порядок ответов
Ожидаемая нагрузка	Заданный профиль входов	Время, память, ошибки
Stress	Нагрузка выше ожидаемой	Предел, отказ, восстановление
Для сервиса	Параллельность запросов	p95/p99, throughput, ошибки

Wheel: проверяем доставку кода

Editable-установка удобна для разработки, но не проверяет содержимое wheel.

Build

Собираем устанавливаемый архив кода

Новое окружение

Устанавливаем зависимости и wheel

Чужая папка

Без PYTHONPATH и доступа через editable

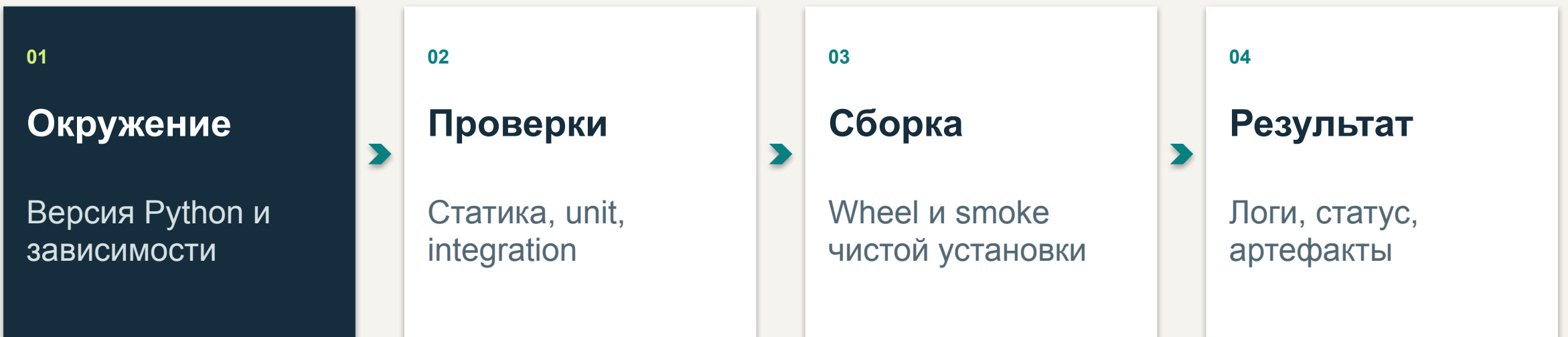
BASH

```
# В starter, зависимости уже установлены
python -m build --wheel --no-isolation
```

```
# В отдельном venv, вне репозитория
python -m pip install --no-deps /abs/pkg.whl
python -c "import taxi_duration; \
print(taxi_duration.__file__)"
taxi-duration --help
```

CI: делаем проверки повторяемыми

Изменение кода запускает те же проверки в подготовленном окружении.



Три независимых условия готовности

Одно зелёное число не заменяет остальные доказательства.

01

Качество

Baseline и модель
сравнены на одной
когорте; ограничения
названы

02

Корректность

Контракты и тесты
пройдены; строки не
теряются

03

Переносимость

Код установлен из wheel;
bundle и запуск понятны
получателю

Практика: ноутбук → три правки → тесты

Среда и данные подготовлены заранее.

01

Notebook

Split, признаки, baseline



02

Три TODO

Duration, fit/transform,
inference



03

Тесты

Свои проверки,
исправления, общий pytest

Практика: CLI → wheel → проверка результата

Результат: код, bundle, метрики, паспорт, CSV и проходящие тесты.

01

CLI

Train и predict; проверяем
файлы и ride_id



02

Wheel

Вместе: установка в среде
получателя



03

Защита

Объяснение результата и
ограничений